Minicurso Docker



Aula 1

COMP PET

UFU Universidade Federal de Uberlândia

facom

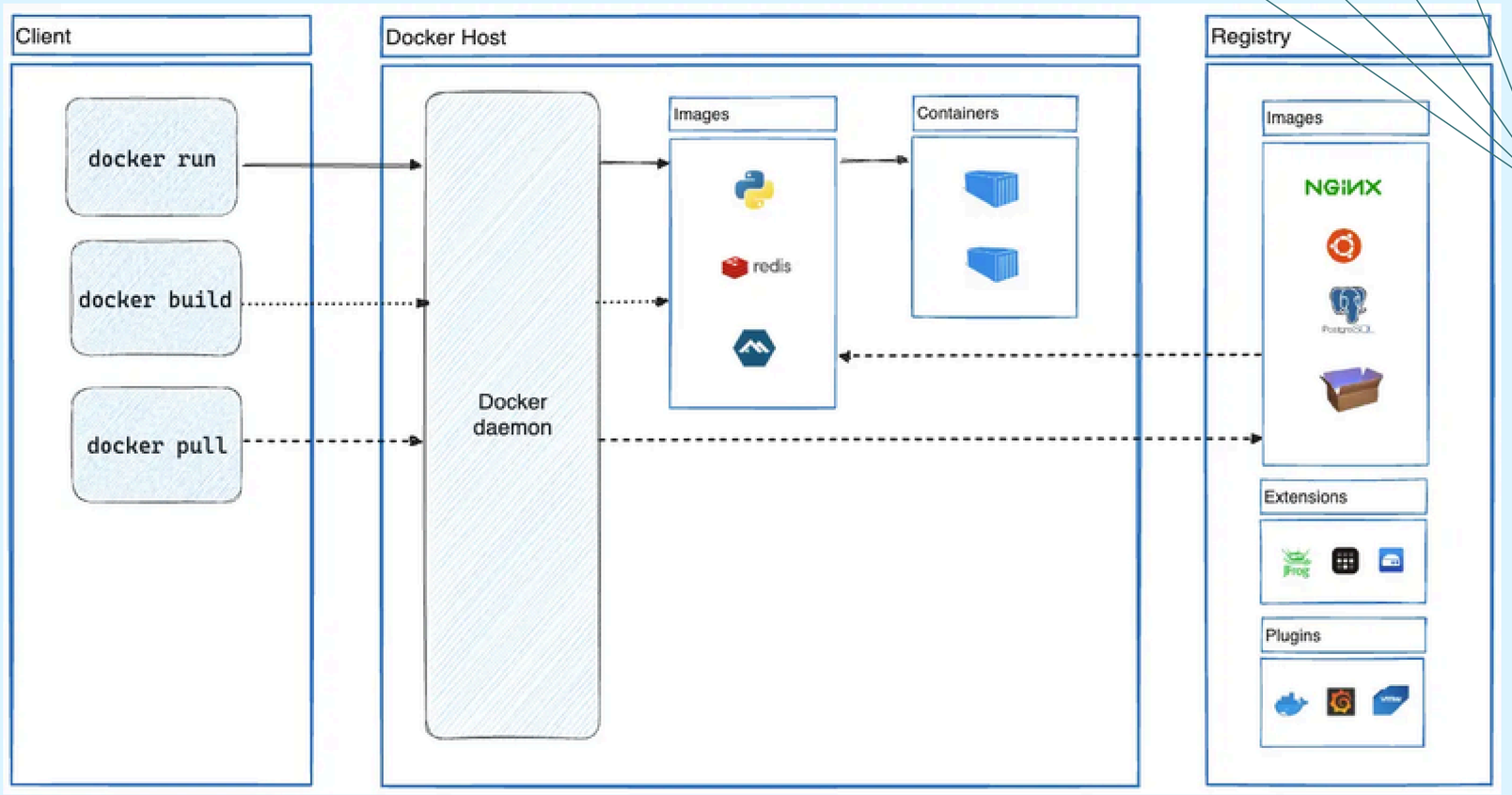
Contêiners?

- Os cointêners empacotam as bibliotecas e dependências de aplicações em um único lugar
- Facilitam o isolamento e compartilhamento de aplicações.
- Contêiner ≠ Máquina virtual!



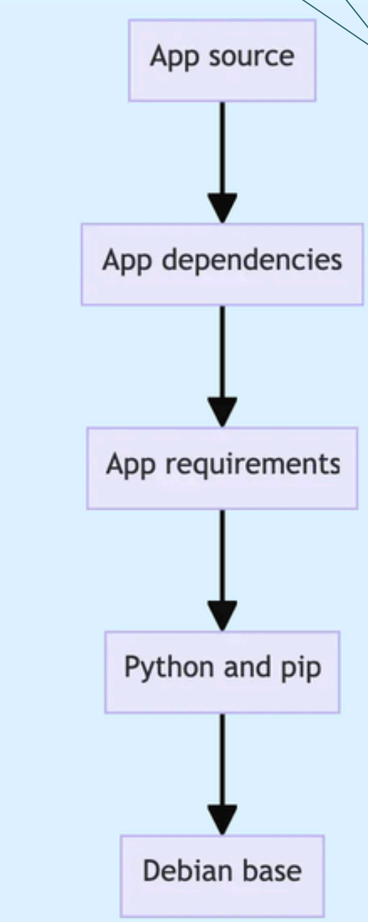
Docker?

Plataforma que permite configurar, executar e gerenciar contêiners



Imagens?

- Os contêiners são distribuidos por meio de imagens
- As imagens contam ao docker os arquivos e bibliotecas de um contêiner.
- Imagens são compostas por camadas imutáveis de arquivos



Imagens x contêiners

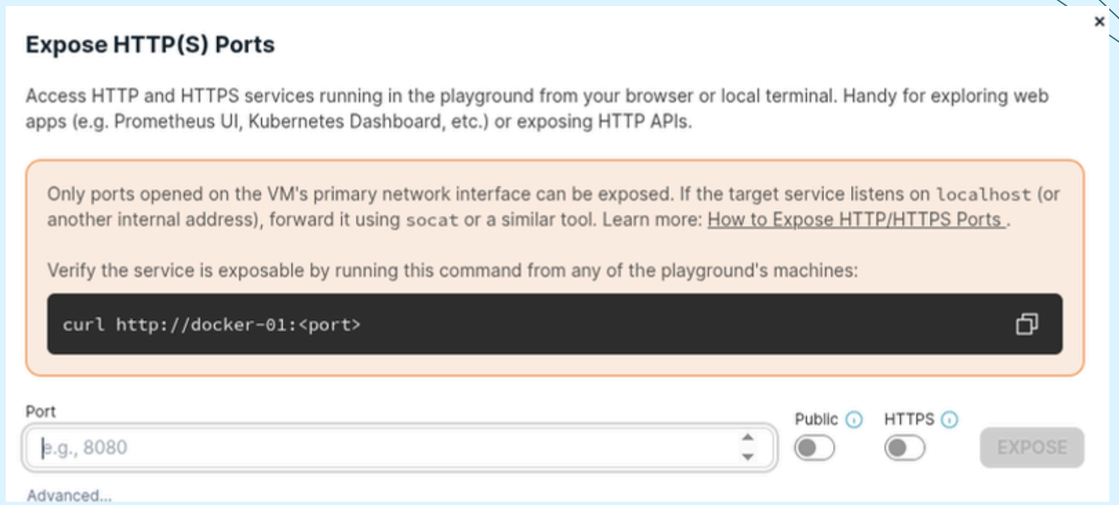
- No geral, um contêiner é uma instância em execução de uma imagem.
- Docker tem uma plataforma própria para o compartilhamento de imagens: https://hub.docker.com/
- Imagens são definidas por um *dockerfile*

Comandos

- O Docker é usado principalmente por meio de comandos no terminal
- Vamos ver alguns dos mais importantes
- Para testar sem precisar baixar:
- https://labs.iximiuz.com/playgrounds/docker
- https://killercoda.com/docker

Exemplos

- Para ver e modificar exemplos:
- *git clone https://github.com/comppet-ufu/minicurso-docker*
- O iximiuz permite expor portas para o público (necessário para o primeiro exemplo)



Comandos básicos

- docker pull *img*: baixa uma imagem do dockerhub
- docker run *img*: começa um contêiner
  - -it: permite enviar comandos para o contêiner
  - -d: roda o serviço no plano de fundo
  - -v: permite acesso do conteiner a um diretório no host
  - -p: Abre uma das portas para o contêiner
- docker start *cont*: volta a executar um contêiner já criado (note que é diferente do run).

Comandos básicos

- docker stop *cont*: para de executar um contêiner
- docker ps: lista os contêiners em execução
    - -a: lista também os contêiners parados
- docker rm: remove um contêiner.
- *dica:* A maioria dos comandos aceita uma flag *--help*, que mostra uma descrição do comando e suas opções. O melhor jeito de aprender é explorando!

Na prática

- Tente começar o contêiner *docker/welcome-to-docker* ou *hello-world*
- Use *docker ps* para ver que está ativo
- Use a flag *-p* para ligar as portas
- Pare o contêiner.

Comandos do dockerfile

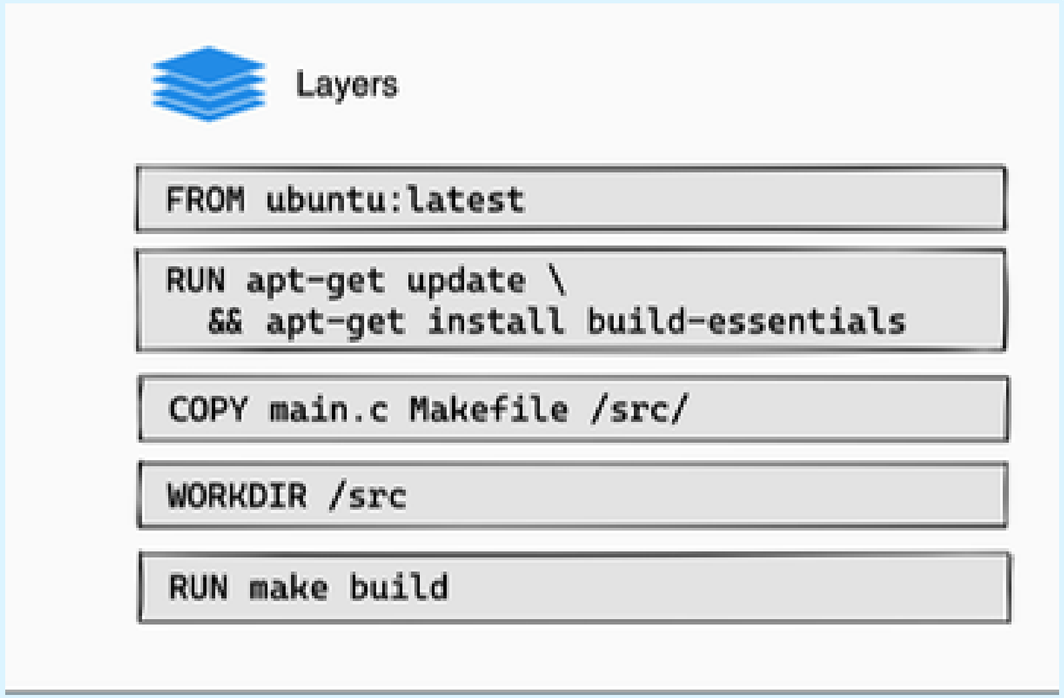
- FROM: indica a imagem base
- WORKDIR: Troca o diretório dos próximos comandos
- COPY: cópia arquivos do host para o contêiner
- RUN: executa um comando
- ENTRYPOINT: Programa executado quando o contêiner inicia

Mais comandos importantes

- CMD: Comandos passados quando ao contêiner quando inicia
- EXPOSE: configura uma porta para ligar ao docker (igual à flag *-p*)
- USER: muda o usuário que roda os comandos.
- *dockerfiles* também podem ser separadas em diferentes fases, diminuindo o tamanho das imagens

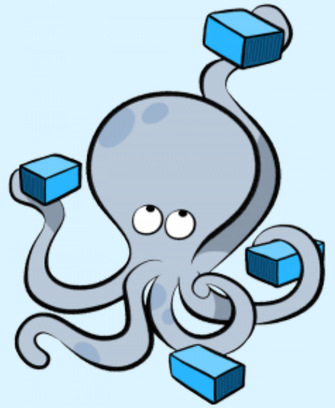
Para construir a imagem

- Com o dockerfile pronto, basta rodar *docker buildx build* para criar a imagem em seu sistema.
- Cada comando do dockerfile é uma camada



Construindo sistemas maiores

- Vimos o básico de contêiners e como usá-los.
- Para aplicações mais complexas, costuma-se separar cada “parte” em um contêiner.
- O Docker oferece um meio de gerenciar isso => *docker compose*
- Além disso, e se precisarmos de persistência de dados?
- Veremos isso e mais um pouco na próxima aula!



Muito obrigado!



Dúvidas?

COMP PET

UFU Universidade Federal de Uberlândia

facom